

Einführung in die Programmierung mit Python

HSLU Hochschule
Luzern

Syllabus

1. Python installieren; Jupyter Notebooks
2. Grundlagen: Arithmetik, Variablen und Datentypen
3. Methoden; Listen; Bedingte Anweisungen
4. Schleifen
5. Benutzerdefinierte Funktionen
6. **Datenaufbereitung und Grafische Darstellungen**

Python-Pakete

- Der Funktionsumfang von Python lässt sich mit sogenannten *Paketen* erweitern.
- Viele davon sind in Anaconda bereits vorinstalliert. Wir müssen sie nur laden (*importieren*):

```
In [ ]: import numpy # Paket für numerische Berechnungen. Siehe optionales Notebook
```

Nun kann man Funktionen aus dem Paket wie folgt verwenden:

```
In [ ]: numpy.mean([1, 2, 3])
```

Um sich Tipparbeit zu ersparen, kann man beim Import einen Kurznamen angeben. Dieser ist willkürlich, aber für häufig genutzte Pakete haben sich Konventionen ergeben, z.B. `np` für numpy:

```
In [ ]: import numpy as np
        np.mean([1, 2, 3])
```

Es ist auch möglich, bestimmte Funktionen aus einem Paket zu importieren, so dass man sie direkt verwenden kann:

```
In [ ]: from numpy import mean
        mean([1, 2, 3])
```

Pandas Dataframes

- `pandas` (von *panel data*) ist eines der wichtigsten Pakete ([Benutzerhandbuch](#)).
- Es bietet eine Reihe von Datenstrukturen (*series*, *dataframes* und *panels*), die für die Speicherung von Beobachtungsdaten entwickelt wurden, sowie leistungsstarke Methoden zur Manipulation (*munging* oder *wrangling*) dieser Daten.
- Es wird üblicherweise als `pd` importiert:

```
In [ ]: import pandas as pd
```

- Pandas ist unglaublich leistungsfähig; wir werden hier nur die Oberfläche ankratzen. In der Praxis sind LLMs nützlich, um jeweils die richtige Syntax zu finden.
- Eine Excel-Datei können wir wie folgt einlesen:

```
In [ ]: nvda_data = pd.read_excel('data/nvda.xlsx', index_col='Date')
```

- Hinweis: die Datei `nvda_data.xlsx` wurde mit folgendem Code erstellt. Er verwendet das Paket `yfinance`, mit dem sich bequem Daten von Yahoo Finance beziehen lassen:

```
#!/pip install yfinance
import yfinance as yf
from datetime import datetime, timedelta
end_date = datetime.now()
start_date = end_date - timedelta(days=5*365) # siehe unten
nvda_data = yf.download('NVDA', start=start_date, end=end_date)
nvda_data.Close.to_excel("nvda.xlsx")
```

- Nützliche Methoden zur Gewinnung von Zusammenfassungsinformationen über einen Dataframe sind `mean`, `std`, `info`, `describe`, `head` und `tail`.

```
In [ ]: nvda_data.head() # zeigt die ersten paar Zeilen; data.tail zeigt die letzten paar
```



```
In [ ]: nvda_data.describe()
```


- Fügen wir dem Dataframe eine neue Spalte mit den Tagesrenditen hinzu:

```
In [ ]: nvda_data['Return'] = nvda_data['NVDA'].pct_change()  
nvda_data.head()
```

- Wie wir sehen, enthält der erste Eintrag für Return fehlende Daten (Frage: Warum?).
- Dies ist ein häufiges Problem. Die Methode `dropna` kann verwendet werden, um Zeilen mit fehlenden Werten zu löschen:

```
In [ ]: nvda_data.dropna(inplace=True)  
nvda_data.head()
```

Plotten

Pandas kann direkt zum Plotten verwendet werden. Erweiterte Funktionen erfordern `matplotlib` (mehr dazu unten).

```
In [ ]: nvda_data.NVDA.plot()
```

- Erweitertes Plotten erfordert die Bibliothek `matplotlib` ([Benutzerhandbuch](#)), die von den Plotting-Funktionen von Matlab® inspiriert ist.
- Die Haupt-Plotting-Funktionen befinden sich im Modul `pyplot`. Es wird üblicherweise importiert als

```
In [ ]: import matplotlib.pyplot as plt
```

`matplotlib` ermöglicht es uns, den obigen Plot zu customizen:

```
In [ ]: nvda_data.NVDA.plot()  
plt.xlabel("Date")  
plt.ylabel("Price")  
plt.gcf().autofmt_xdate()  
plt.show()
```

- Die Bibliothek `seaborn` ([Benutzerhandbuch](#)) bietet statistische Visualisierungen auf höherem Niveau:

```
In [ ]: #!/conda install -c conda-forge -y seaborn  
import seaborn as sns  
sns.set_style("whitegrid") # Kosmetik  
sns.set_palette("viridis") # Kosmetik  
sns.histplot(nvda_data['Return'], bins=30, kde=True, stat="density", alpha=0.6)  
plt.title("Distribution of Returns", fontsize=14, pad=15)  
plt.xlabel("Value", fontsize=12)  
plt.ylabel("Density", fontsize=12)  
plt.tight_layout()  
plt.show()
```

- Ich werde hier nur eine kurze Einführung in matplotlib geben. Das grundlegende Objekt in matplotlib ist eine `Figure`, in der sich `Subplots` (oder `Axes`) befinden.
- Um eine neue Figure zu erstellen, einen Axis hinzuzufügen und darauf zu plotten:

```
In [ ]: # mit dem Inline-Backend müssen diese im selben Cell sein.  
fig = plt.figure(figsize=(6,3)) # eine neue leere Figure erstellen. Grösse ist optional.  
ax1 = fig.add_subplot(121) # Layout: (1x2). ax1 ist der obere linke  
ax2 = fig.add_subplot(122)  
ax1.plot(range(10))  
ax2.plot(range(10, 0, -1));
```

- Standardmässig plottet matplotlib in den aktuellen Axis und erstellt einen (sowie eine Figure), falls nötig. Mit der Bequemlichkeitsmethode `subplot` können wir dasselbe erreichen, ohne explizit auf Figures und Axes zu verweisen:

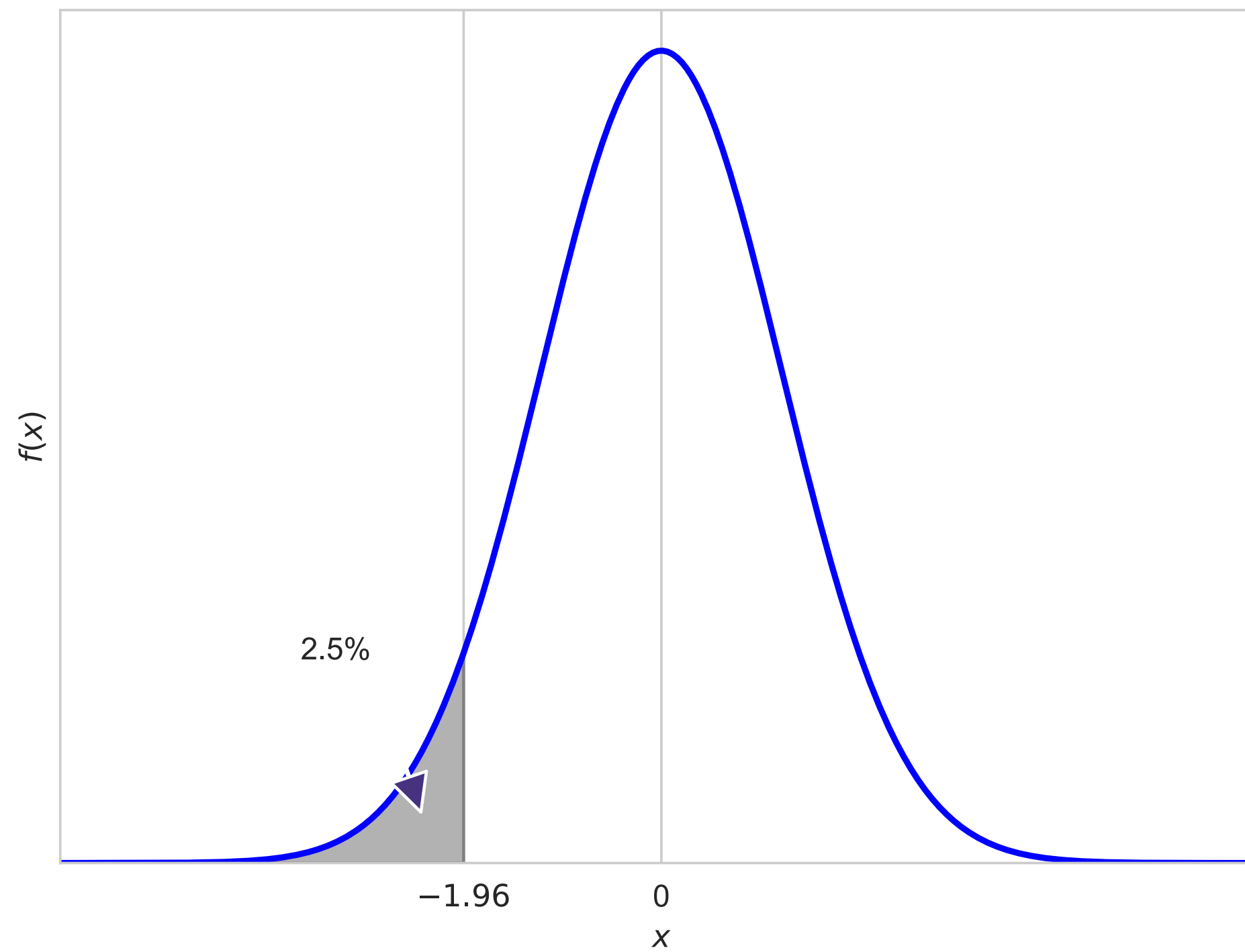
```
In [ ]: plt.subplot(121)
plt.plot(range(10))
plt.subplot(122)
plt.plot(range(10, 0, -1));
```

- Um zwei Vektoren x und y gegeneinander zu plotten:

```
In [ ]: import numpy as np
x = np.linspace(-10, 10, 100)
y = x**2
plt.plot(x,y,'r--') # gestrichelte rote Linie
plt.xlabel('$x$') # Mathematische (LaTeX) Gleichungen können durch Einschliessen in $$ eingefügt werden.
plt.ylabel('$y$')
plt.title('Eine Parabel')
plt.legend(['$f(x)$']) # erwartet eine Liste von Strings
plt.xlim(xmin=-8, xmax=8); # Achsengrenzen
# plt.savefig('filename.pdf') # den Plot auf der Festplatte speichern
```


Ein fortgeschrittenes Beispiel: Quantile der Normalverteilung

```
In [ ]: from matplotlib.patches import Polygon
import scipy.stats as stats
a, b, c = -5, 5, stats.norm.ppf(0.05)
x = np.linspace(a, b, 500)
y = stats.norm.pdf(x)
fig = plt.figure(figsize=(7, 5))
ax = fig.add_subplot(111)
plt.plot(x, y, 'b', linewidth=2)
plt.ylim(ymin=0)
plt.xlim(xmin=a, xmax=b)
Ix = np.linspace(a, c)
Iy = stats.norm.pdf(Ix)
verts = [(a, 0)] + list(zip(Ix, Iy)) + [(c, 0)]
poly = Polygon(verts, facecolor='0.7', edgecolor='0.5')
ax.add_patch(poly)
ax.annotate(
    '2.5%', xy=(-2, 0.025), xytext=(-3, 0.1),
    arrowprops=dict(width=.5),
)
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
ax.set_xticks([c, 0])
ax.set_xticklabels(['$-1.96$', '0'])
ax.set_yticks([])
plt.savefig('img/var.svg')
plt.close()
```

Empfohlene Lektüre

- <https://python-course.eu/numerical-programming/> 23-28, 32-34

Hausaufgaben

https://github.com/guipsamora/pandas_exercises/tree/master/01_Getting_%26_Knowing_Your_Data/Chipotle

(Nur bis Aufgabe 12. Zusätzlich: Visualisieren Sie die Verteilung der Artikelpreise mit Seaborn und/oder Matplotlib. Hinweis: nutzen Sie ein LLM (ChatGPT, Grok, Claude, ...) wenn Sie nicht weiterkommen.

In []:

